

객체지향프로그래밍
최종 보고서

LAPIS MYSTERY

GitHub: <https://github.com/songwoojin6024/LapisMystery>
YouTube: <https://youtu.be/elpt-X7cW40?si=EzlOFro74N4Ni3JR>

팀명: 도원결의
팀장: 송우진(5820493)
팀원: 박보승(5946005), 황효빈(5820752)

목차

◎프로젝트 개요

선정배경 · 벤치마킹 · 개발환경

◎개발 프로그램 설명

UI/UX · 클래스 다이어그램

◎테스트 및 장애 해결

테스트 결과 · 발생 문제 및 해결과정

◎성과 및 결론

기대효과 ·구현 완료/미완료

◎개발 일정 및 업무 분장

주차별 일정 · 팀원 역할

◎프로젝트 개요

■ 작품 주제 선정 배경

사고력과 판단력을 요구하는 장르로 단순 조작 게임보다 더욱 몰입도가 높고, 학교라는 친숙한 배경속에서 현실감있게 플레이 할 수 있으며, 단순 추리를 넘어 여러 미니게임을 추가해 플레이어의 참여도를 극대화 할 수 있어서 추리 어드벤처 게임을 선택하였습니다.

■ 벤치마킹

- 단간론파: 추리 어드벤처 게임
- 역전재판: 범정 배틀 게임

■ 개발환경

- 개발언어: c++
- IDE/OS: Visual Studio /Window 10
- 사용 라이브러리: Window APL(conio.h, windows.h, iostream, vector, string, ctime, array, iomanip)
STL(vector, string)
- 버전관리: Git/GitHub
- Window 콘솔환경(터미널), 200X50 터미널 크기

◎개발 프로그램 설명

■ UI/UX 화면 구조

-게임은 Window 터미널에서 동작하며 화면은 맵 출력 영역과 하단 상태바를 출력합니다.

- 조작키는 WASD와 방향키를 모두 지원합니다.

- ▲ 또는 W: 위로 이동
- ▼ 또는 S: 아래로 이동
- ◀ 또는 A: 왼쪽 이동
- ▶ 또는 D: 오른쪽 이동

I: 인벤토리 열기

Enter: 선택 및 진행

ESC: 상점, 인벤토리, 대화 화면 나가기

- 플레이어 화면에서는 플레이어의 방향에 따라 ▲, ▼, ◀, ▶ 기호가 표시됩니다.
이를 통해 현재 플레이어가 어느 방향을 바라보고 있는지 직관적으로 확인할 수 있습니다.

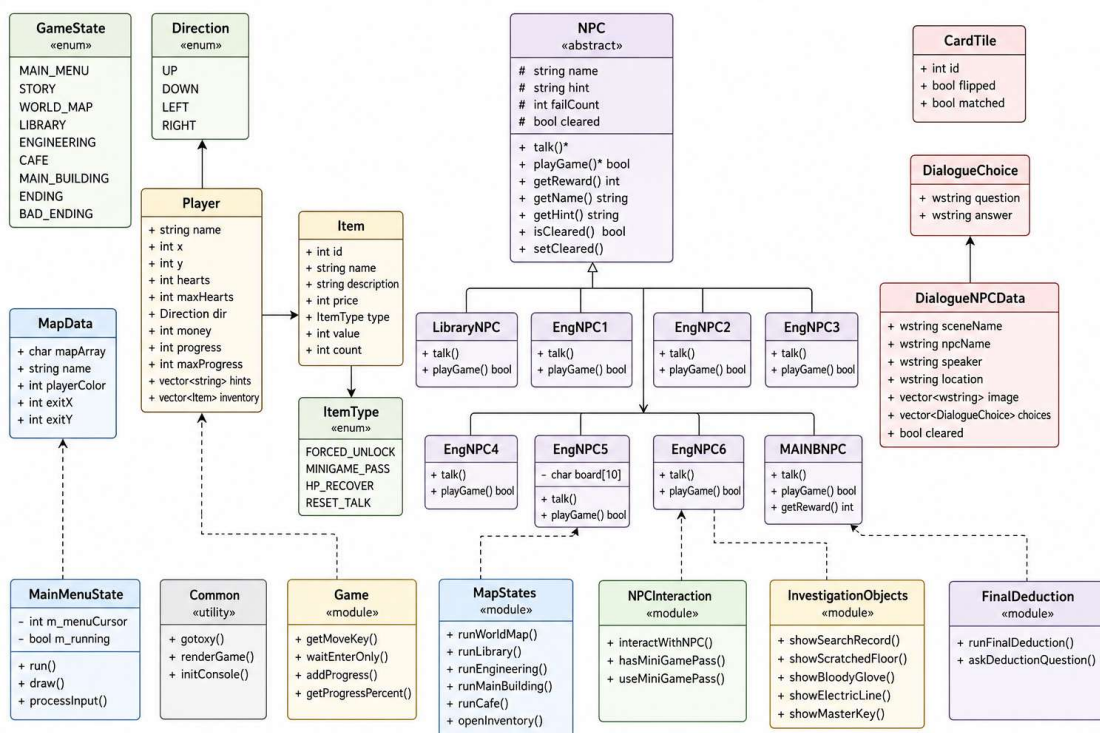
- 하단 상태바에는 현재 플레이어의 상태 정보가 표시됩니다.
표시 항목은 체력(♥/♡), 보유 금액, 수집한 단서 수, 현재 좌표, 진행률, 인벤토리 안내 (II 가방)로 구성되어 있습니다.
이를 통해 플레이어는 현재 수사 진행 상황과 보유 자원을 즉시 확인할 수 있습니다.
- NPC 대화 화면은 게임 NPC와 대화 전용 NPC로 구분된다.
게임 NPC와 접촉하면 NPC의 대사 출력 후 미니게임이 시작되며, 미니게임 성공 시 단서와 보상을 획득할 수 있습니다.
대화 전용 NPC 화면은 NPC 이미지, 대사창, 선택지, 오른쪽 단서 목록으로 구성되어 있으며, 플레이어가 선택지를 고르면 NPC의 답변을 확인할 수 있습니다.
- 미니게임 화면은 NPC별로 서로 다르게 구성된다.
카드 맞추기, 가위바위보, 사다리타기, 숫자 게임, 틱택토, 행맨, 업다운 번호 맞추기 등의 미니게임을 제공하며, 실패 시 재도전 또는 체력 감소가 발생합니다.
또한 인벤토리에 뇌물용 초콜릿이 있을 경우 미니게임을 1회 패스할 수 있습니다.
- 상점 화면은 카페 지역에서 진입할 수 있으며, 아이템 이름, 가격, 보유량, 효과 설명을 표 형태로 보여줍니다.
플레이어는 보유 금액을 사용하여 단서 획득 아이템, 미니게임 패스 아이템, 체력 회복 아이템을 구매할 수 있습니다.
- 인벤토리 화면은 I 키를 눌러 열 수 있으며, 현재 보유 중인 아이템과 효과를 확인할 수 있습니다.
일부 아이템은 사용 시 단서를 추가하거나 체력을 회복하며, ESC 키를 눌러 다시 맵 화면으로 돌아갈 수 있습니다.
- 스토리 화면은 게임 시작 시 사건의 배경을 텍스트와 ASCII 아트로 출력합니다.
오프닝은 일정 시간 간격으로 대사가 출력되며, 사용자가 원할 경우 스킵할 수 있도록 구성하였습니다.
- 최종 주리 화면은 플레이어가 충분한 단서를 모은 뒤 본관에서 진입할 수 있습니다.
플레이어는 여러 주리 질문에 답하고 최종 범인을 지목한다.
정답을 선택하면 굿엔딩으로, 오답을 선택하거나 체력이 모두 소진되면 베드엔딩으로 진행됩니다.

■ 객체지향 요소

	클래스		내용
추상클래스	NPC		talk(), playGame()을 순수 가상 함수(= 0)로 선언. 직접 인스턴스화 불가, NPC 공통 인터페이스 정의
상속	LibraryNPC > NPC		도서관 NPC, talk()/ playGame()구현
	EngNPC1 > NPC		공학관 NPC1, talk()/ CardGame()구현
	EngNPC2 > NPC		공학관 NPC2, talk() / playGame() 구현. 가위바위보 미니게임 실행
	EngNPC3 > NPC		공학관 NPC3, talk() / playGame() 구현. 사다리타기 미니게임 실행
	EngNPC4 > NPC		공학관 NPC4, talk() / playGame() 구현. 숫자 부르기 게임 실행
	EngNPC5 > NPC		공학관 NPC5, talk() / playGame() 구현. 틱택토 미니 게임 실행
	EngNPC6 > NPC		공학관 NPC6, talk() / playGame() 구현. 행맨 미니게임 실행
	MAINBNPC > NPC		본관 경비실 금고 NPC, talk() / playGame() 구현. 업다운 번호 맞추기 게임 실행
오버라이딩 (다형성)	LibraryNPC, EngNPC1~5, MAINBNPC		talk() 함수 : 각 NPC마다 다른 대화 내용 출력
			playGame() 함수: NPC마다 다른 미니게임 실행
다형성	NPC* engNpcs[]		부모 클래스 포인터 배열을 이용하여 여러 NPC 객체를 하나의 방식으로 관리
	interactWithNPC(NPC * npc, Player& p)		NPC 부모 포인터를 매개변수로 받아, 어떤 NPC든 동일한 함수에서 대화/미니게임/보상 처리
캡슐화	NPC	접근제어자	name, hint, failCount, cleared는 자식 클래스에서만 접근 가능
		protected	
	CardGame	private	board, cursorPos, tries, matchedCnt는 외부 노출 차단
	MainMenu State	private	m_menuCursor, m_running은 내부 상태만 관리

클래스	Player	플레이어 이름, 위치, 체력, 돈, 진행도, 단서 목록, 인벤토리 관리
	MapData	맵 배열, 맵 이름, 출구 좌표 등 지역 이동에 필요한 데이터 관리
	DialogueNPCData	대화 NPC의 장면명, NPC 이름, 화자, 위치, 이미지, 선택지 데이터 관리

■ 클래스 다이어그램



NPC 추상 클래스를 기반으로 각 NPC 클래스가 상속되는 구조로 설계하였습니다.

NPC는 talk()와 playGame()을 순수 가상 함수로 선언하여 공통 인터페이스 역할을 하며, 각 NPC는 이를 오버라이딩하여 서로 다른 대화와 미니게임을 실행합니다.

- NPC(추상): talk(), playGame() 순수 가상 함수 선언
- LibraryNPC: 도서관 NPC, 닌센스 퀴즈 구현
- EngNPC1~6: 공과대학 NPC, 카드 맞추기·가위바위보·사다리타기·숫자 게임·틱택토·행맨구현
- MAINBNPC: 본관 금고 NPC, 업다운 번호 맞추기 게임 구현
- Player: 위치, 하트, 돈, 진행도, 단서, 인벤토리 관리
- MapData: 맵 배열, 맵 이름, 출구 좌표 관리
- Item: 상점 및 인벤토리 아이템 정보 관리
- DialogueNPCData: 대화 전용 NPC의 장면, 이미지, 선택지 데이터 관리

■ 핵심 기능

-interactWithNPC()

NPC* 하나를 받아 모든 NPC와의 상호작용을 공통처리합니다. 최대 3번 도전을 할 수 있고, 성공 시 힌트 + 금액, 실패 시 하트를 감소합니다

-runWorldMap()

L/G/S/B 타일 접촉 시 GameState를 전환하여 해당 건물 내부로 이동합니다. WASD와 화살표 키를 모두 지원합니다.

-renderGame()

gotoxy(0,0)으로 커서를 초기화 후 전체 맵을 출력하고, 플레이어 위치에 방향 기호(▲▼◀▶)를 색상과 함께 덮어씁니다.

-openInventory()

플레이어가 보유한 아이템 목록을 출력하고 아이템 사용을 처리한다. 일부 아이템은 단서를 추가하고, 회복 아이템은 하트를 회복합니다.

-runFinalDeduction()

수집한 단서를 바탕으로 최종 추리를 진행한다. 여러 추리 질문을 통과한 뒤 범인을 지목하며, 정답 시 굿엔딩, 오답 시 베드엔딩으로 분기됩니다.

-runCafe()

카페 상점 화면을 출력하고 아이템 구매를 처리한다. 플레이어는 단서 획득 아이템, 미니게임 패스 아이템, 하트 회복 아이템을 구매할 수 있습니다.

■ 객체지향 요소 정리

- 주상 클래스: talk()와 playGame()을 순수 가상 함수(=0)로 선언하여 직접 인스턴스화 불가, 인터페이스 역할 수행
- 상속(LibraryNPC, EngNPC): 구체 NPC 클래스가 NPC를 public 상속, 공통멤버(name, hint, failCount, cleared) 재사용
- 다형성(interactWithNPC()): NPC* 하나로 LibraryNPC, EngNPC1~6, MAINBNPC 등 다양한 NPC 객체를 동일한 방식으로 처리
- 오버라이딩(NPC::talk): 부모 클래스 NPC의 순수 가상 함수를 각 NPC 클래스에서 재정의 하여 NPC마다 다른 대화와 미니게임을 실행하도록 하였다.
- 캡슐화(NPC::name, hint, failCount, cleared): protected로 선언하여 외부 직접 접근을 제한하고, getName(), getHint(), isCleared(), setCleared() 등의 접근 함수를 통해 필요한 값만 사용하도록 하였다.
- 클래스 활용(Player, MapData, Item): Player는 플레이어 상태와 단서/인벤토리를 관리하고, MapData는 맵 정보와 출구 좌표를 관리하며, Item은 상점 및 인벤토리 아이템 정보를 관리한다.

◎테스트 및 검증

■ 테스트 결과

- 스토리 화면 → 월드맵 전환(S키 입력): 정상
- 오프닝 스킵 기능: 정상
- 한글 이름 입력 후 이동: 정상
- WASD / 방향키 이동: 정상
- I 키 인벤토리 열기: 정상
- L/G/S/B 타일 접촉 시 도서관/공과대학/카페/본관 입장: 정상
- 건물 내부에서 월드맵 복귀: 정상
- NPC 미니게임 3회 도전: 정상
- 미니게임 성공 시 단서 및 금액 획득: 정상
- 미니게임 3회 실패 시 하트 감소: 정상
- 미니게임 클리어/실패 후 재방문 처리: 정상
- 뇌물용 초콜릿 사용 시 미니게임 패스: 정상
- 조사 오브젝트 접촉 시 단서 획득: 정상
- 단서 목록 및 진행률 표시: 정상
- 상점 아이템 구매: 정상
- 인벤토리 아이템 사용: 정상
- 하트 회복 아이템 사용: 정상
- 하트 0 이하 처리 및 베드엔딩: 정상
- 최종 추리 성공 시 굿엔딩: 정상
- 최종 추리 실패 시 베드엔딩: 정상

■ 주요 장애요인 및 해결과정

- 아스키 아트, 맵을 만드는 과정에서 화면 깨짐 발생
→ 콘솔 크기를 고정하여 줄 밀림 방지
- NPC 재방문 시 이미 클리어하였지만 다시 게임 진행
→ isCleared()함수를 추가하여 재진행하지 않도록 수정
- 건물내부에서 나갈 때 엉뚱한 위치로 스폰되는 버그
→ MapData 클래스에 exitX,exitY 좌표 추가하여 해당 좌표로 복귀 하도록 수정
- 한글 이름 입력 후 WASD 이동이 정상적으로 동작하지 않는 문제 발생
→ getch() 중심 입력 방식에서 GetAsyncKeyState() 기반 입력 방식으로 변경하여 한글 입력 상태에서도 이동이 가능하도록 수정

- 단서 문장이 길어 대화 NPC 오른쪽 단서창을 벗어나는 문제 발생
→ 단서 문자열을 일정 길이로 축약하여 표시하는 기능을 추가.
- 상점 및 인벤토리 화면에서 이동키 문자가 입력창에 남는 문제 발생
→ cin 입력 대신 `_getch()` 기반 메뉴 선택 방식으로 변경하여 불필요한 문자 입력을 방지

◎현재까지 성과 및 결론

■ 기대효과 및 활용방안

- OOP 개념을 실제 동작 프로그램에 적용하는 경험 습득 · NPC 추상화 구조를 통해 구체 클래스로 기능 확장 가능
- NPC 추상 클래스와 상속 구조를 활용하여 새로운 NPC와 미니게임을 쉽게 추가할 수 있는 구조로 만들어 게임 확장에 편리.
- 콘솔 기반 게임이지만 맵 이동, 대화, 단서 수집, 상점, 인벤토리, 최종 주리, 엔딩 분기 등 다양한 게임 요소를 구현.
- 파일을 분할함으로써 쉽게 수정이 가능하고 피드백 수용에 용이
- 향후 새로운 맵, NPC, 단서, 미니게임을 추가하여 더 확장된 주리 어드벤처 게임으로 발전시킬 수 있다.

■ 프로젝트 성과

- 팀원 간 역할 분담 및 GitHub 협업 진행 경험
- 추상 클래스, 상속, 오버라이딩, 다형성, 캡슐화 등 객체지향 요소를 게임 구조에 적용하여 학습 효과에 도움이 되었다.
- 플레이어 이동, NPC 상호작용, 미니게임, 단서 수집, 상점 및 인벤토리, 최종 주리, 굿엔딩/배드엔딩까지 전체 게임 흐름을 완성.
- 직접 디버깅하며 해결하는 과정에서 문제 해결 능력 향상
- 전체적인 게임 흐름을 구현함으로써 보충이 필요하거나 부족한 파트를 보완할 차후 계획을 쉽게 구상함
- 단순한 콘솔 출력 프로그램을 넘어 플레이어의 선택과 단서 수집에 따라 결과가 달라지는 상호작용형 게임을 구현하였다.

◎개발일정

- 1주차 → 시스템 아키텍처 및 기반 클래스 설계
- 2주차 → 데이터 구조화 및 맵 시스템 구현
- 3주차 → NPC상호작용 및 대화 시스템
- 4주차 → 핵심 미니게임 로직 개발
- 5주차 → 단서 조합 및 엔딩 시스템 통합
- 6주차 → 최적화 및 최종 디버깅

◎팀원 업무 분담

- √송우진: 전체 게임 구조, 맵 상태처리, NPC/단서 시스템, 최종처리 구현
- √박보승: 아스키 아트, 인트로, 오프닝, 엔딩 구현
- √황효빈: 메인화면, 카드 맞추기 미니게임, 인터페이스, 상점, 인벤토리 구현